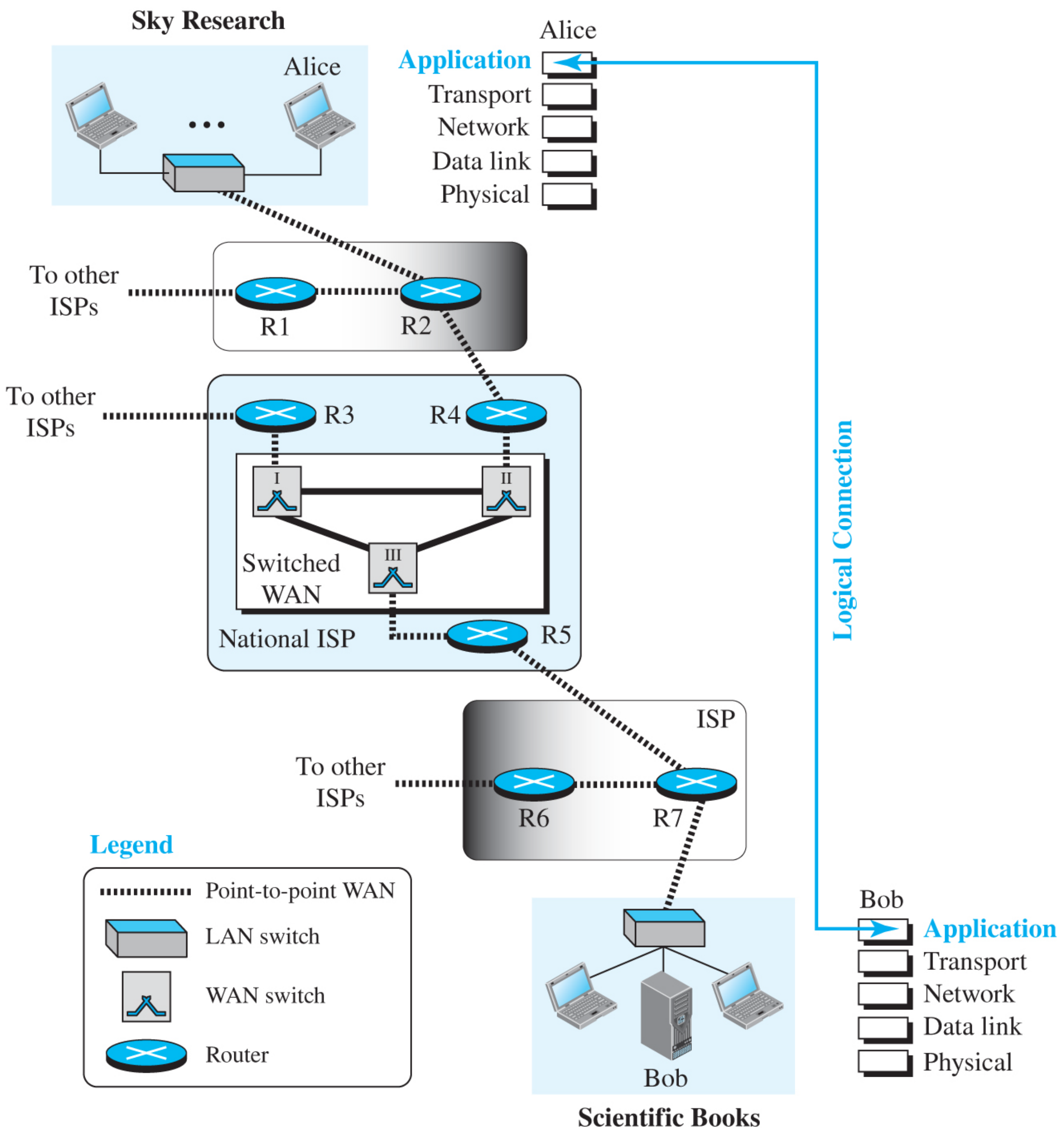
Chapter 10 — Application Layer

- Overview of application-layer services and paradigms.

Introduction

- Provides services to users via logical connections.
- Lower layers are standardized; application protocols may vary.

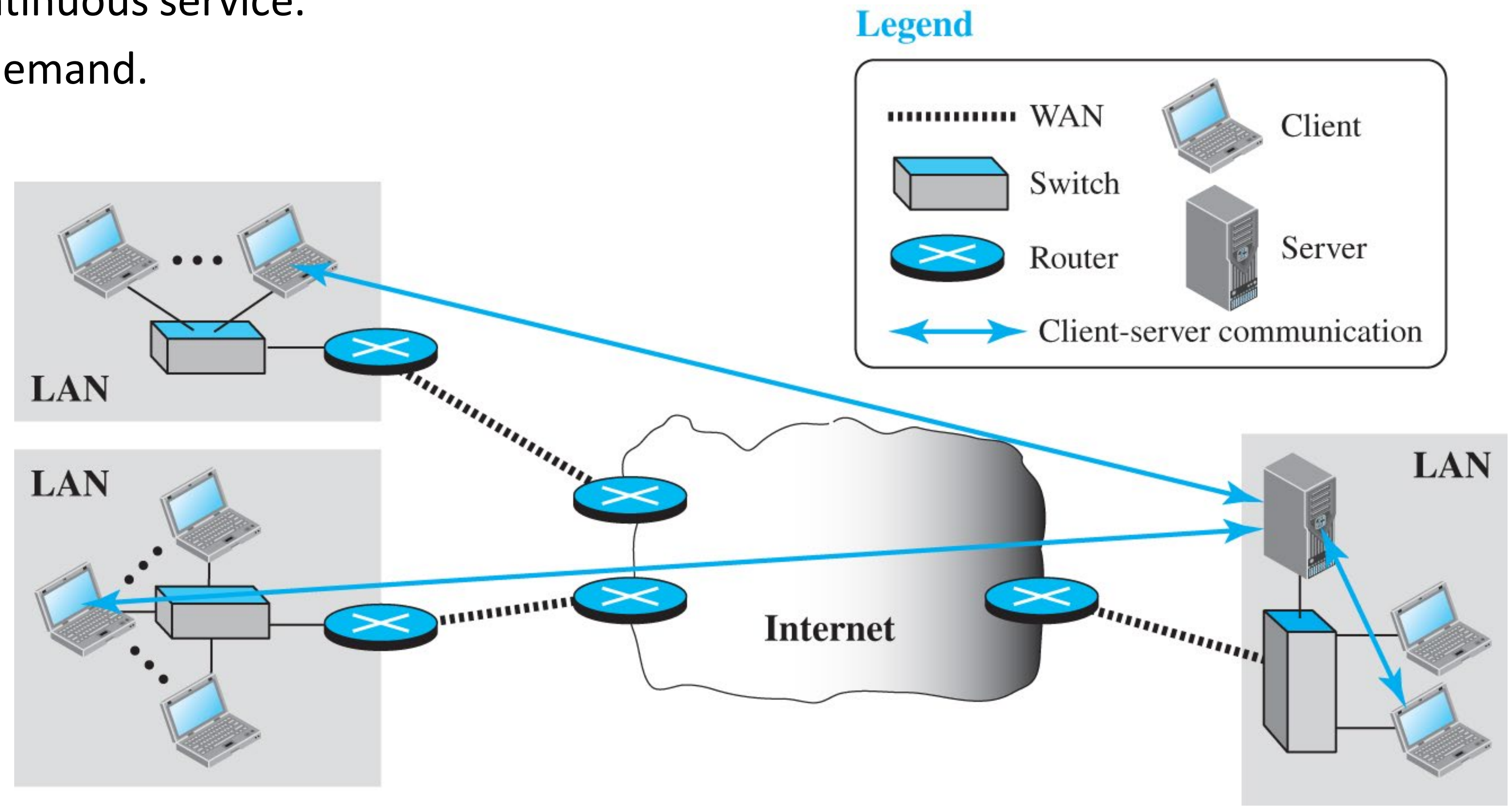

Sky Research
Alice
Alice
Application
Transport
Network
Data link
Physical
To other ISPs
R1
R2
To other ISPs
R3
R4
I
II
III
Switched WAN
National ISP
R5
ISP
To other ISPs
R6
R7
Logical Connection
Legend
Point-to-point WAN
LAN switch
WAN switch
Router
Bob
Scientific Books
Bob
Application
Transport
Network
Data link
Physical

Application-Layer Paradigms

- Two communicating applications across the Internet.
- Main paradigms: Client–Server and Peer-to-Peer.
- Some systems use Mixed approach.

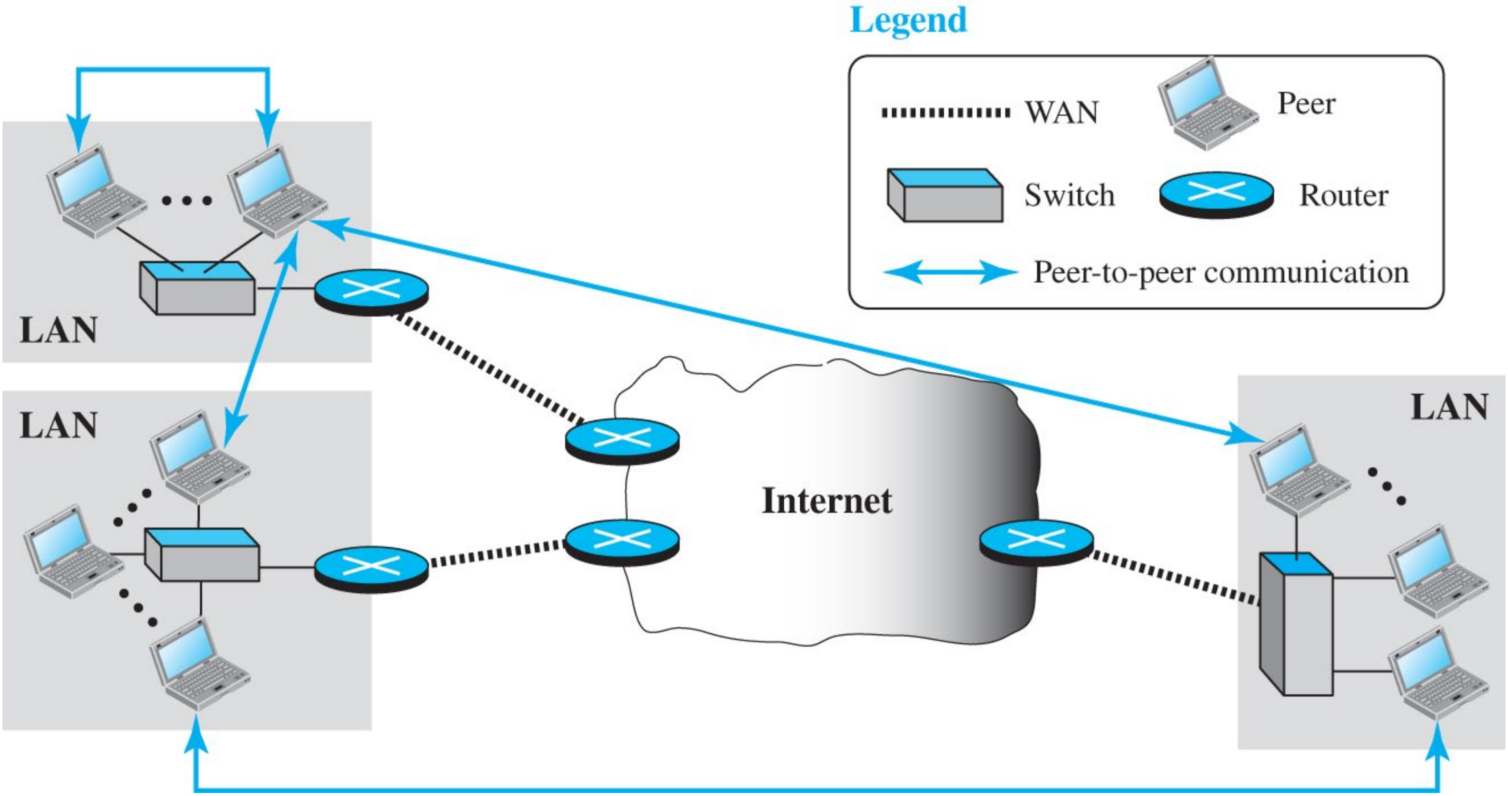
Client–Server Paradigm

- Server process: provides continuous service.
- Client process: initiated on demand.
- Examples: Web, FTP, DNS.



Peer-to-Peer Paradigm

- No permanent server.
- Each peer can act as client and server.
- Examples: BitTorrent, Gnutella.



Mixed Paradigm

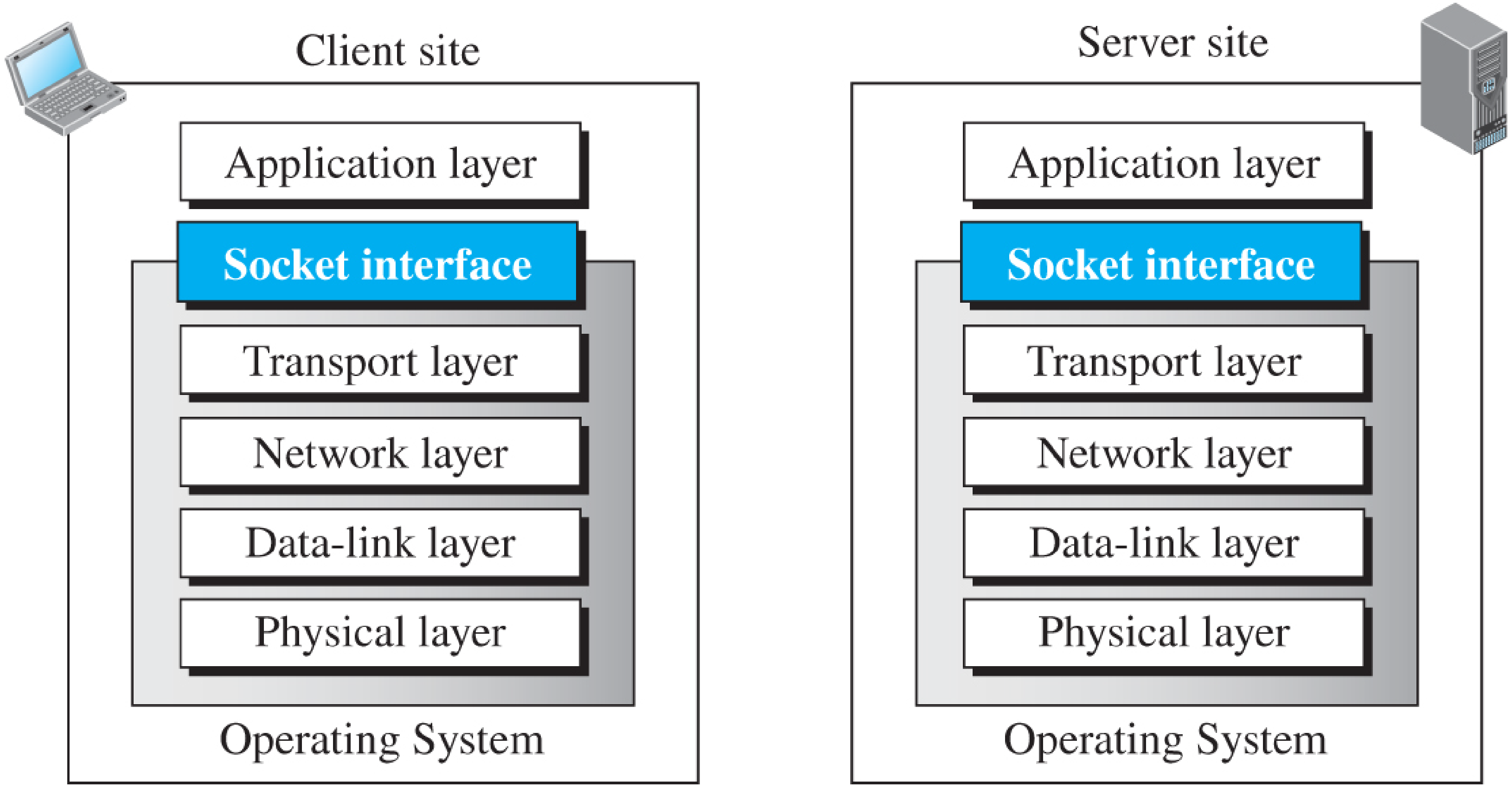
- Combines benefits of both C/S and P2P.
- Client-server for discovery, P2P for data transfer.

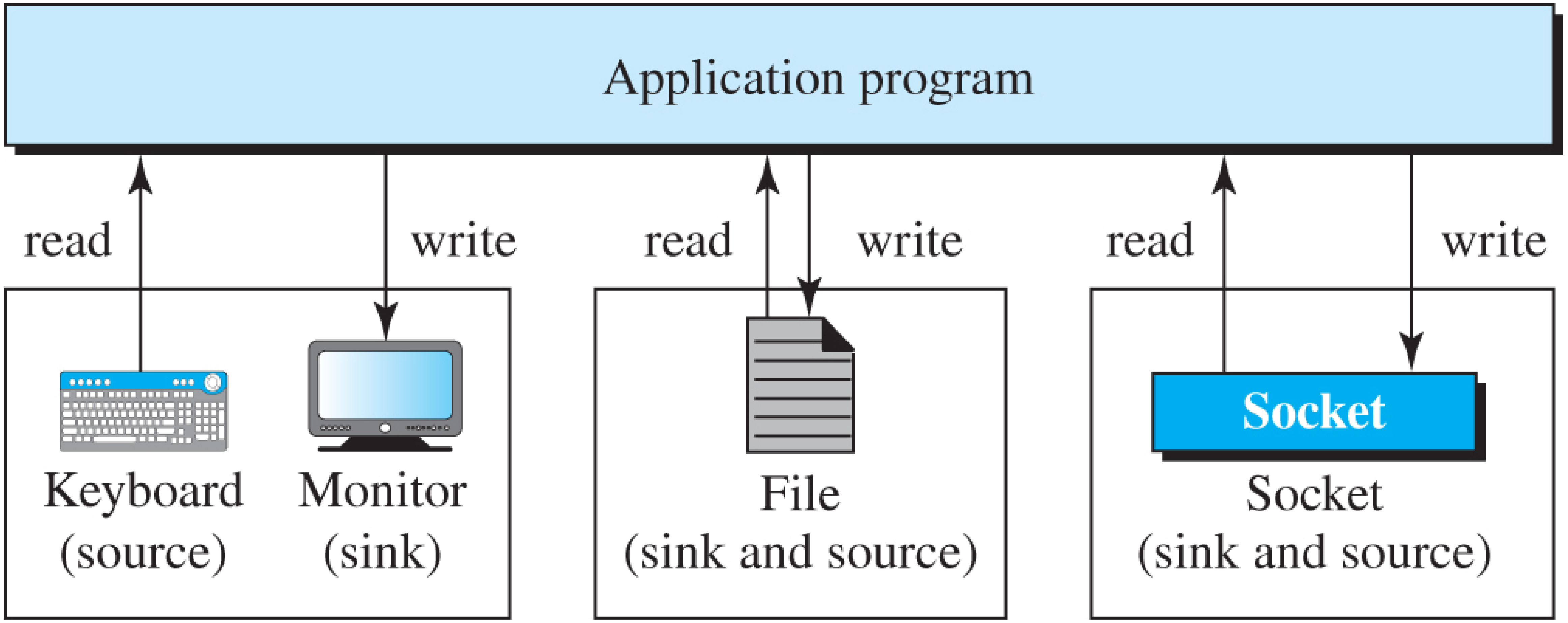
Client–Server Programming

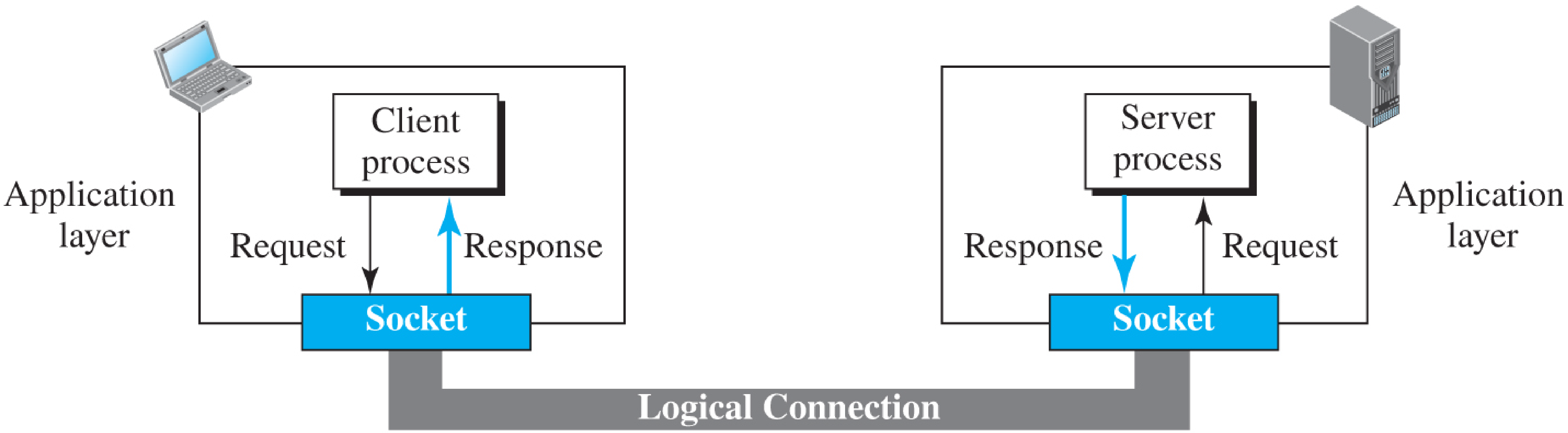
- Communication between two processes.
- Client sends request; server sends response.
- Uses socket API to manage communication.

Socket Concepts

- Socket acts as endpoint for process communication.
- Socket address = IP + Port.
- Each connection uses local and remote addresses.



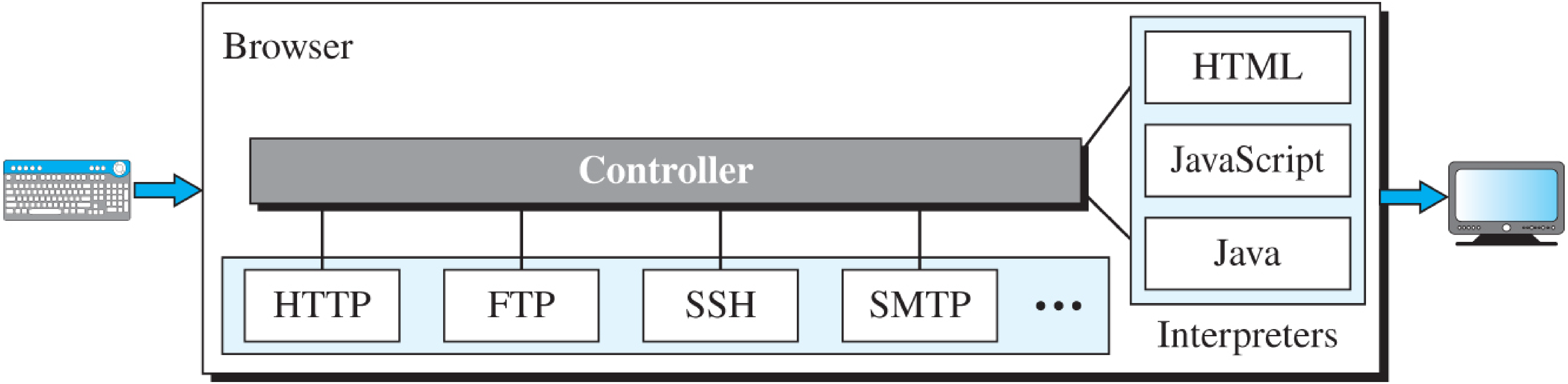

Application program
read
write
read
write
read
write
Keyboard
(source)
Monitor
(sink)
File
(sink and source)
Socket
Socket
(sink and source)



Socket Address

Standard Applications

- Common protocols: HTTP, E-mail, DNS.
- Each provides specific Internet services.

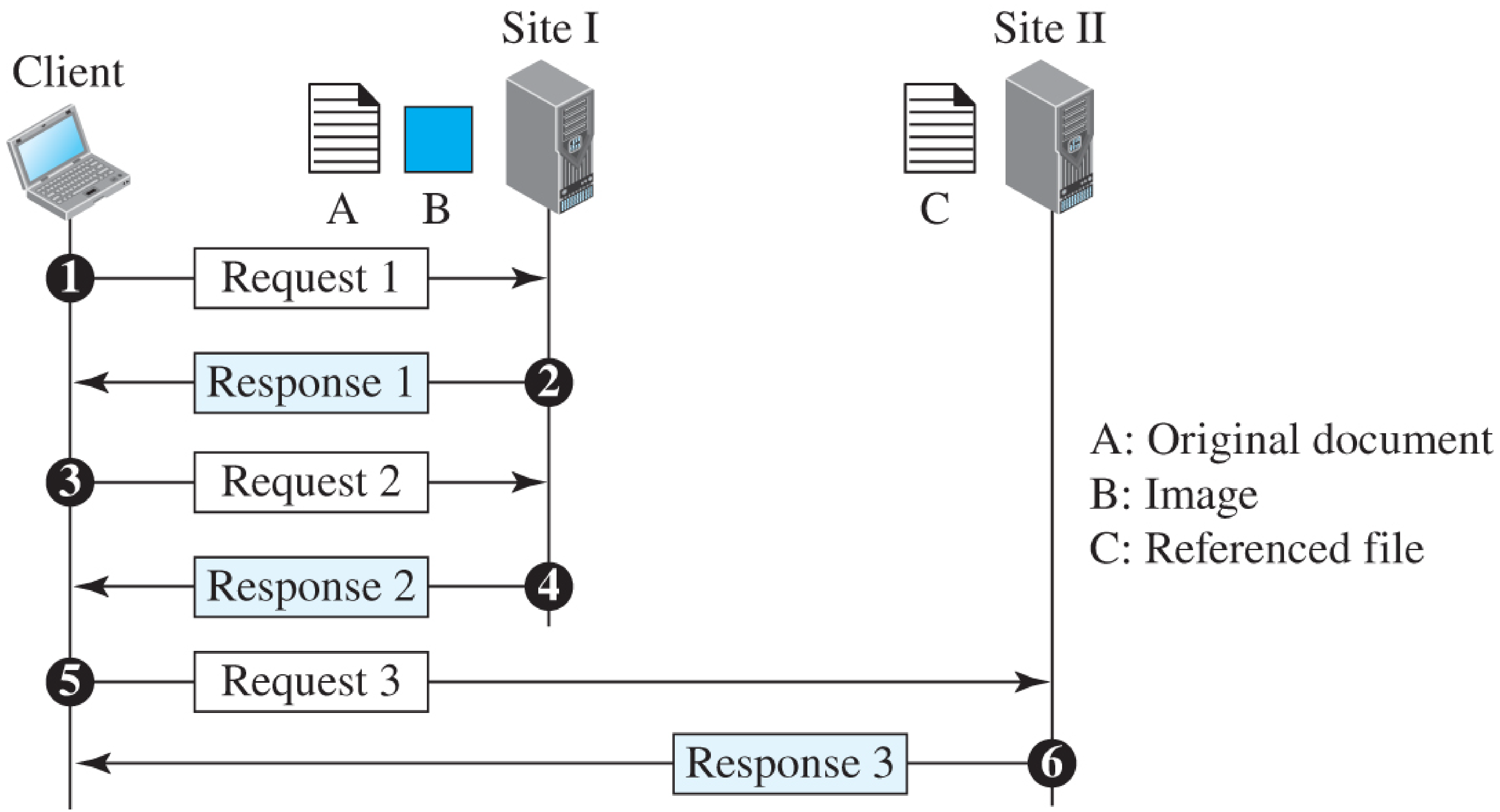


World Wide Web (WWW)

- Distributed system of linked documents.
- Uses HTTP for communication.
- URL identifies each resource.

HTTP Basics

- Browser = client; Web server = server (port 80).
- Non-persistent vs persistent connections.



HTTP Methods

- GET, HEAD, POST, PUT, DELETE, OPTIONS.
- Define the action client requests from server.

Method	*Action*
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

HTTP Headers

- Request headers: User-Agent, Accept, Host.
- Response headers: Server, Content-Type, Set-Cookie.

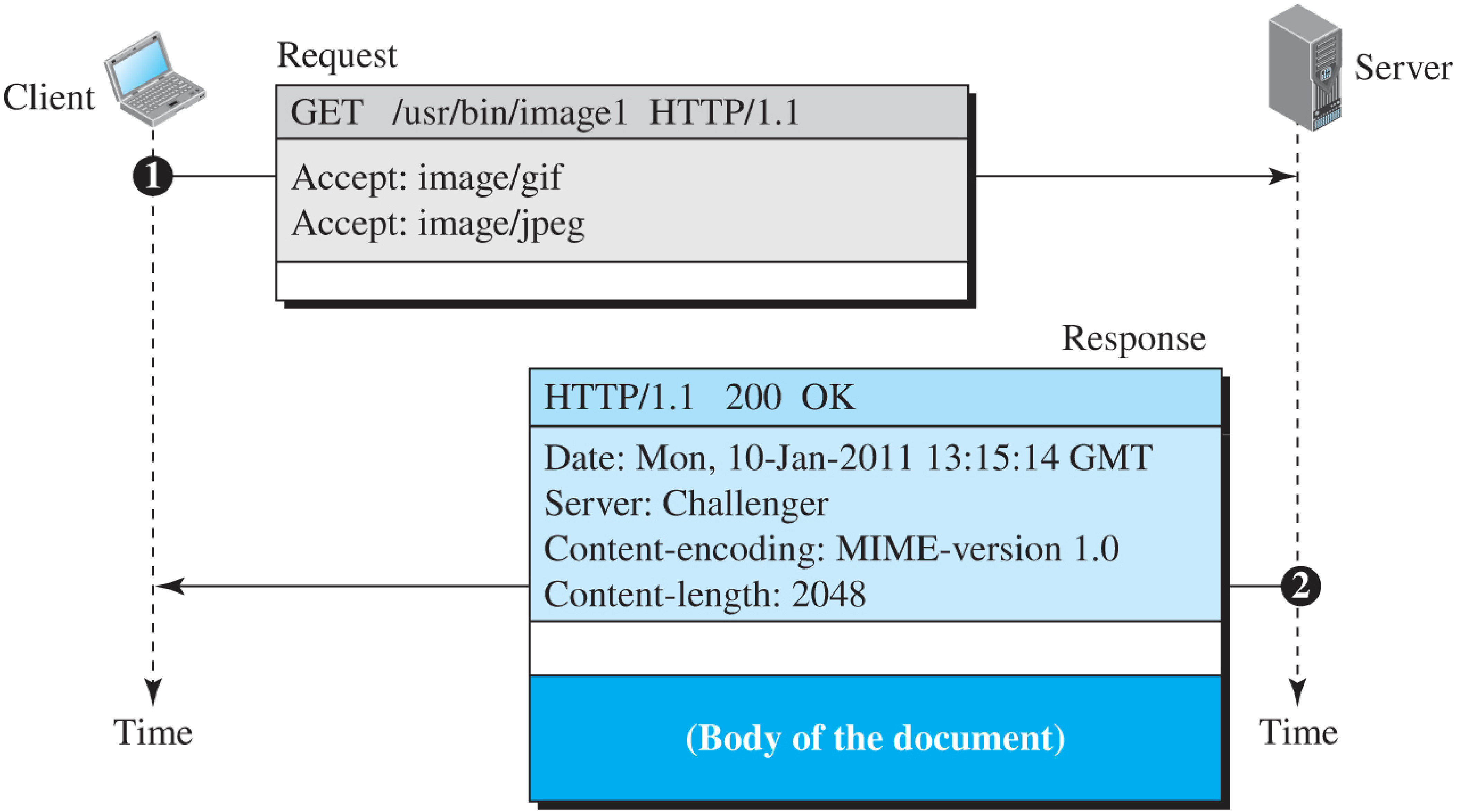
Table 10.2 Request header names

Header	*Description*
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server (explained later in this section)
If-Modified-Since	Specifies if the file has been modified since a specific date

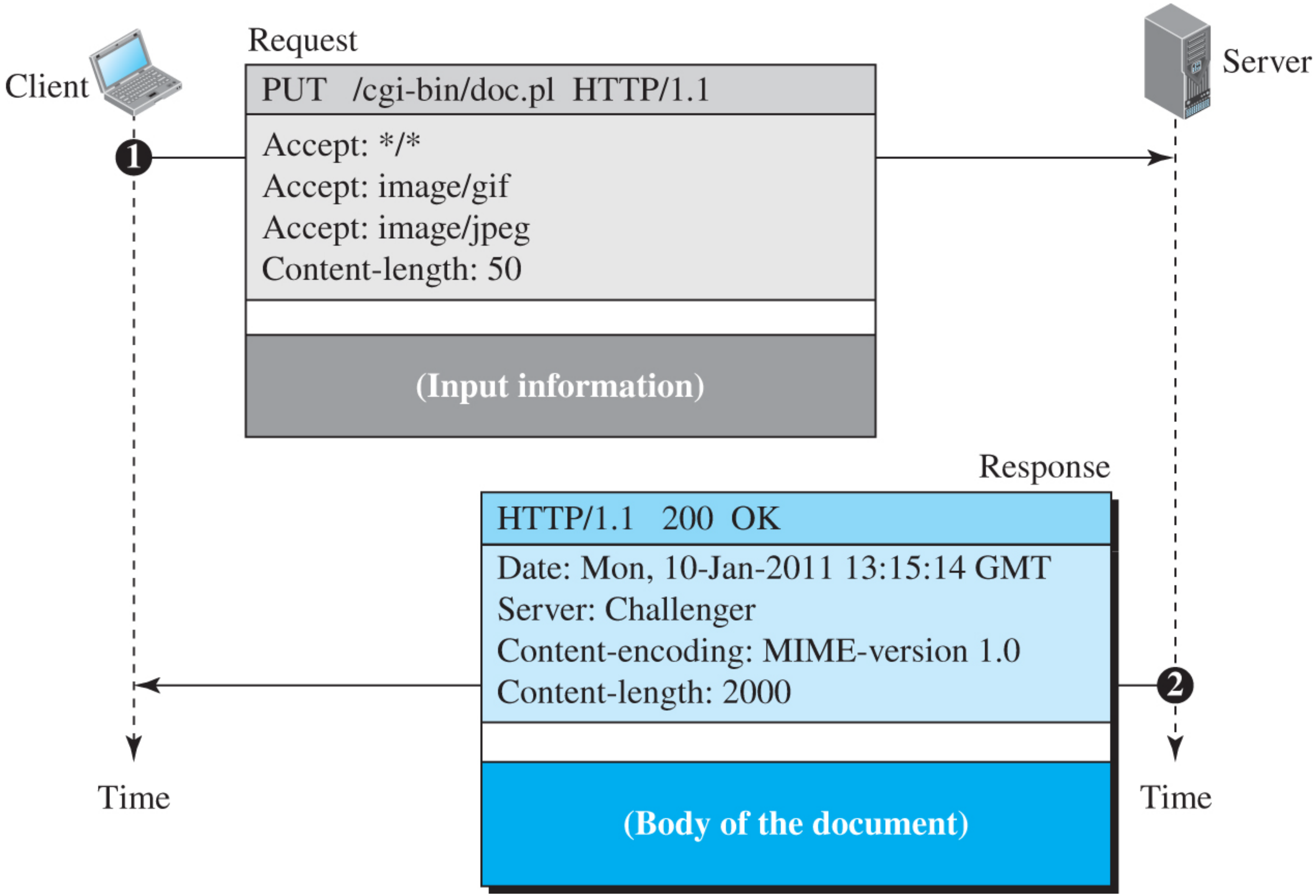
Table 10.3 Response header names

Header	*Description*
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

retrieves a document

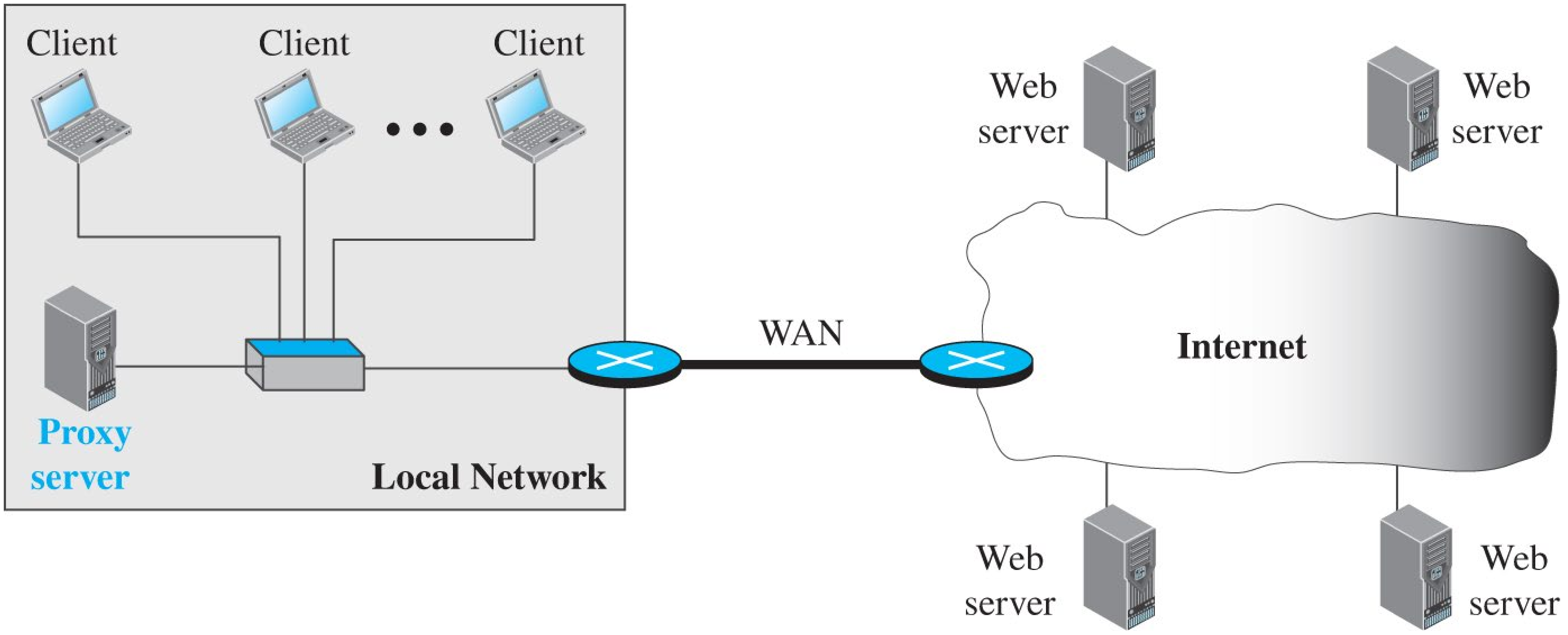


send a web page to be posted on the server



Proxy and Caching

- Proxy caches web pages to reduce load and improve performance.



Cookies

- Small data files stored on client side.
- Used for sessions, preferences, carts.

HTTPS

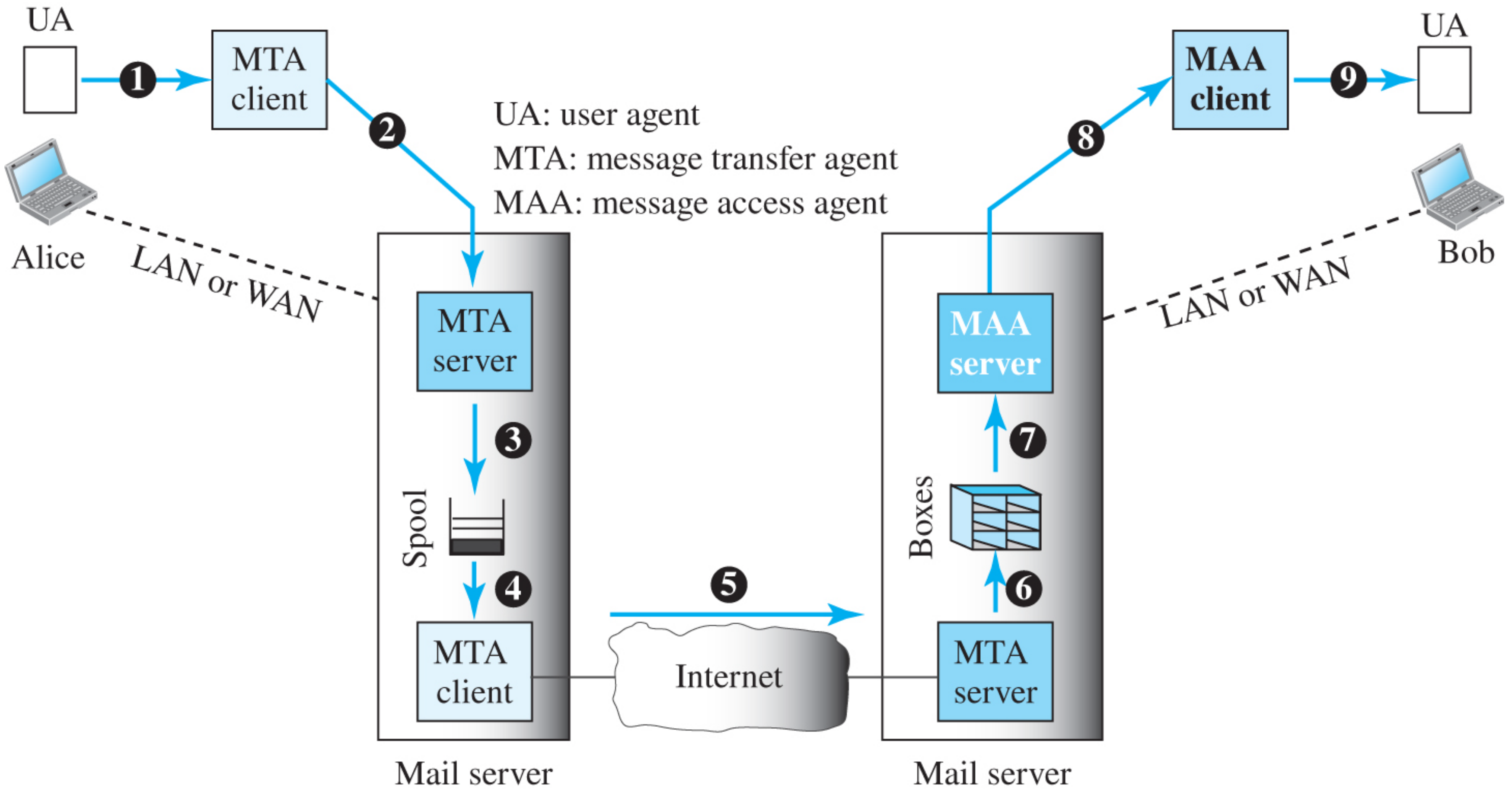
- HTTP over SSL/TLS.
- Provides confidentiality, integrity, authentication.

Electronic Mail

- Enables users to exchange messages.
- One-way transaction between sender and receiver.

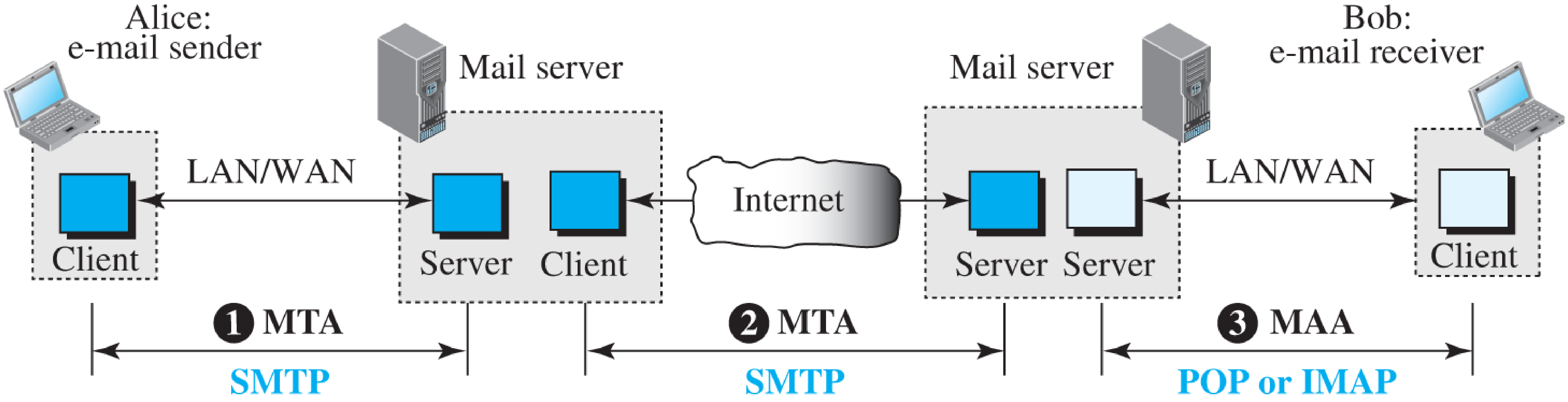
E-Mail Architecture

- Components: User Agent, MTA (SMTP), MAA (POP/IMAP).



SMTP

- Push protocol over TCP port 25.
- Transfers messages between mail servers.

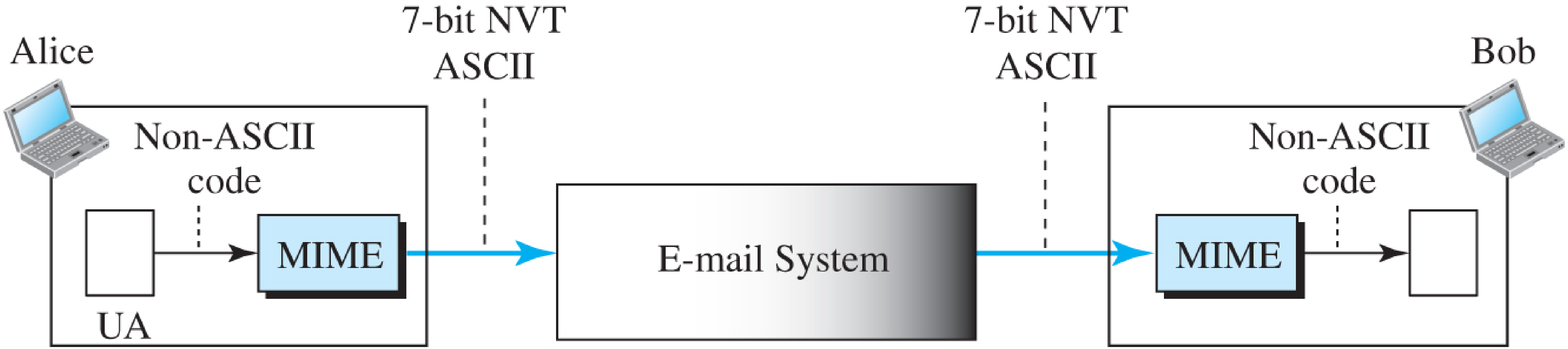


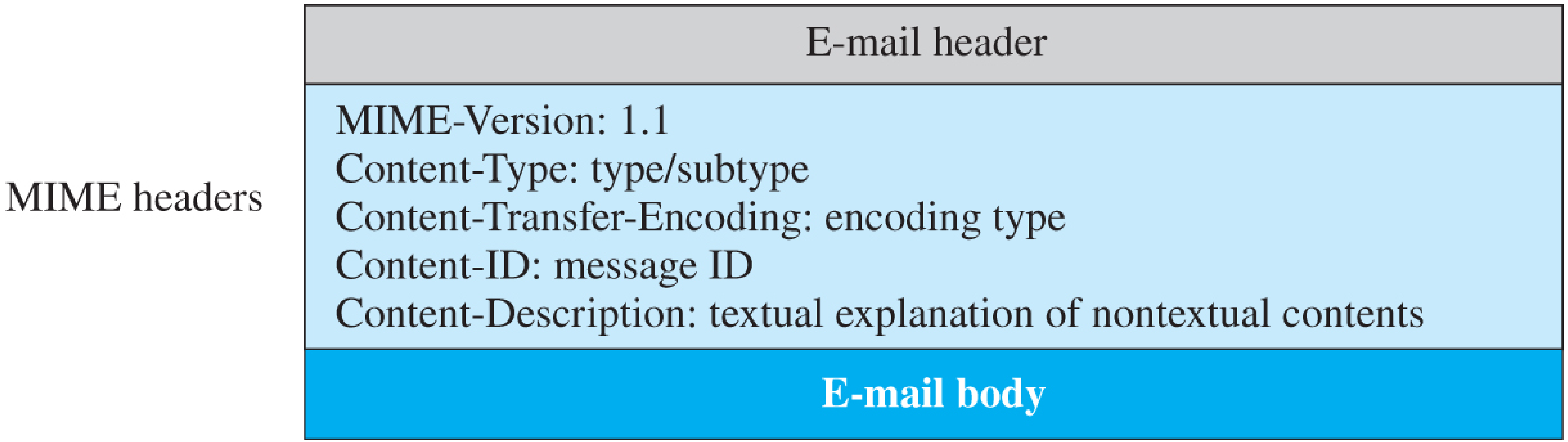
POP3 and IMAP

- Used by clients to pull messages.
- POP3: download & delete.
- IMAP: remote management.

MIME

- Extends e-mail to support multimedia content.
- Allows non-ASCII data (images, audio, video).




MIME headers
E-mail header
MIME-Version: 1.1
Content-Type: type/subtype
Content-Transfer-Encoding: encoding type
Content-ID: message ID
Content-Description: textual explanation of nontextual contents
E-mail body

Data types and subtypes in MIME

Type	*Subtype*	*Description*
Text	Plain	Unformatted
	HTML	HTML format (see Appendix C)
Multipart	Mixed	Body contains ordered parts of different data types
	Parallel	Same as above, but no order
	Digest	Similar to Mixed, but the default is message/RFC822
	Alternative	Parts are different versions of the same message
Message	RFC822	Body is an encapsulated message
	Partial	Body is a fragment of a bigger message
	External-Body	Body is a reference to another message
Image	JPEG	Image is in JPEG format
	GIF	Image is in GIF format
Video	MPEG	Video is in MPEG format
Audio	Basic	Single channel encoding of voice at 8 kHz
Application	PostScript	Adobe PostScript
	Octet-stream	General binary data (8-bit bytes)

Base64 Encoding

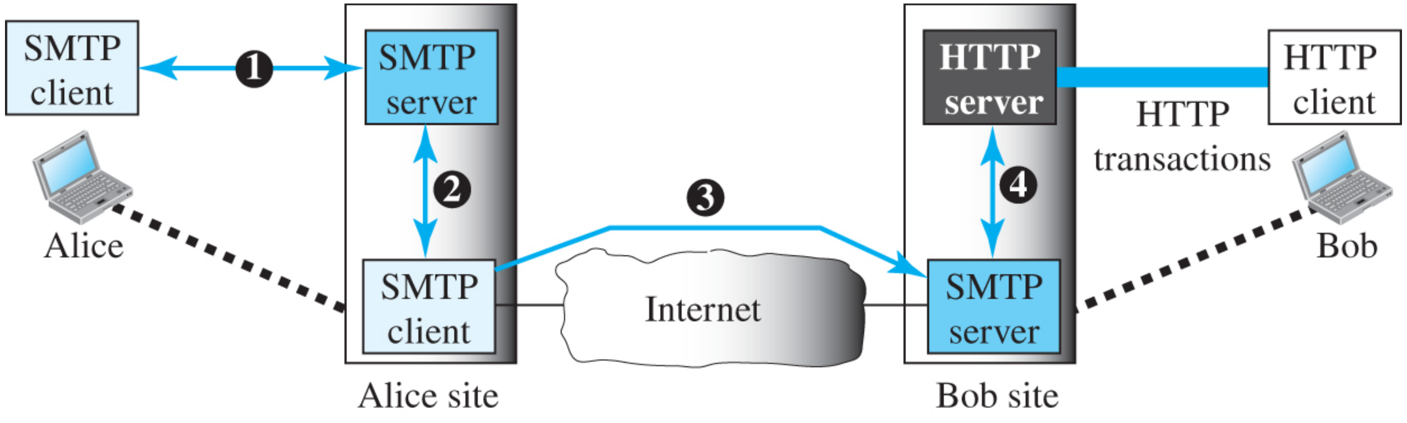
- Converts binary to ASCII (3 bytes → 4 chars).

Methods for content-transfer-encoding

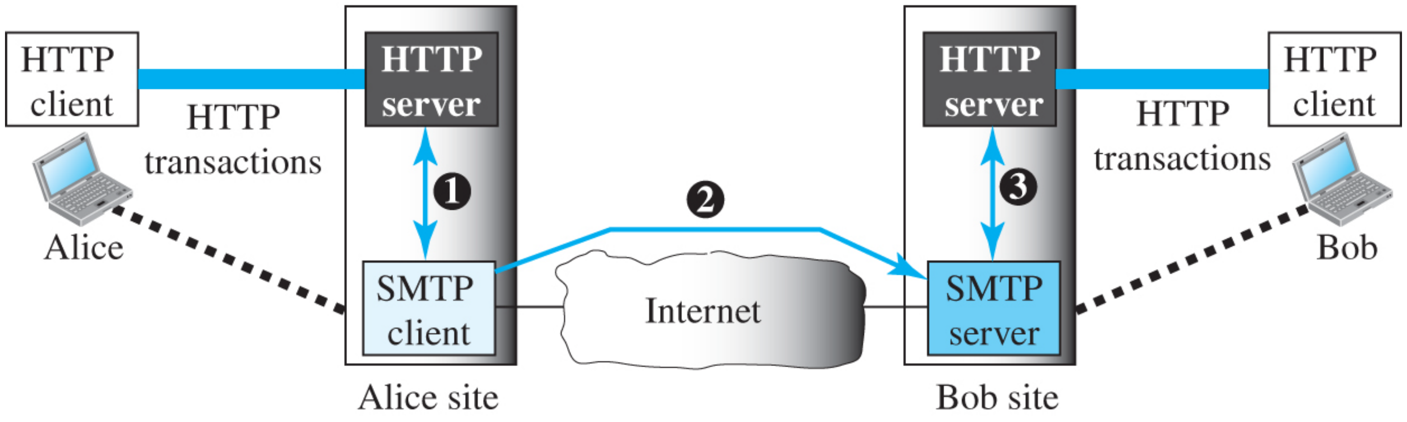
Type	*Description*
7-bit	NVT ASCII characters with each line less than 1000 characters
8-bit	Non-ASCII characters with each line less than 1000 characters
Binary	Non-ASCII characters with unlimited-length lines
Base64	6-bit blocks of data encoded into 8-bit ASCII characters
Quoted-printable	Non-ASCII characters encoded as an equal sign plus an ASCII code

Web-Based Mail

- Services via browsers (Gmail, Yahoo).
- Operate through HTTP gateways.



Case 1: Only receiver uses HTTP



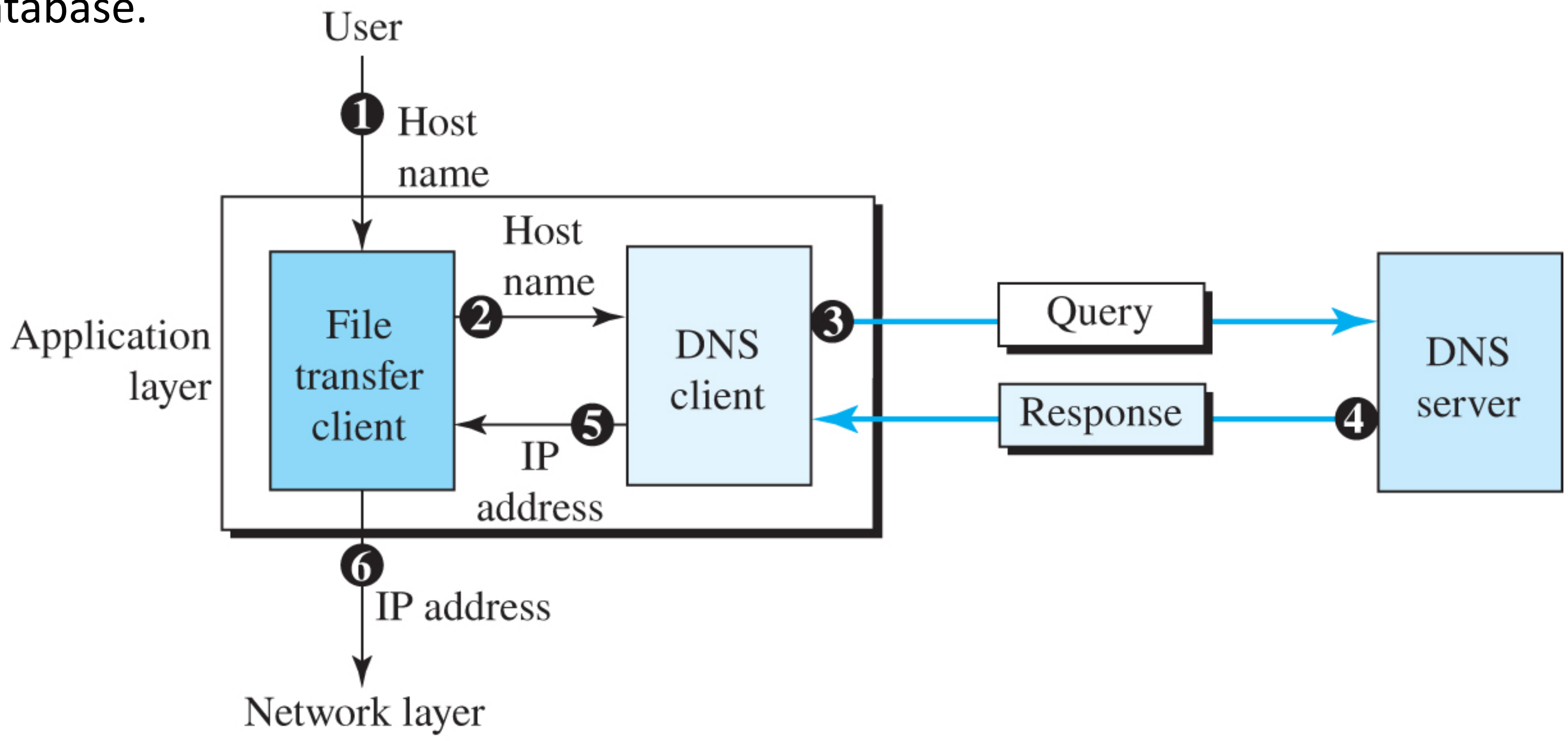
Case 2: Both sender and receiver use HTTP

E-Mail Security

- Standard e-mail insecure.
- Secure via PGP or S/MIME.

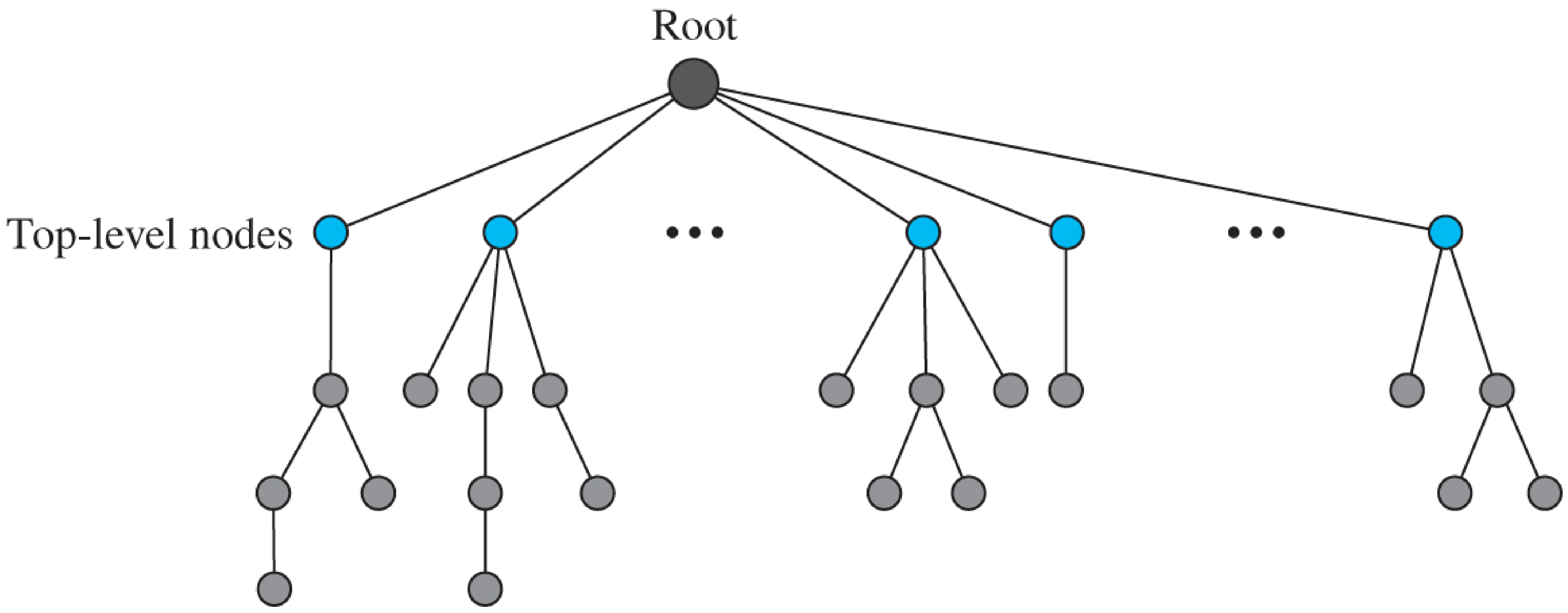
DNS Overview

- Maps names to IP addresses.
- Hierarchical distributed database.



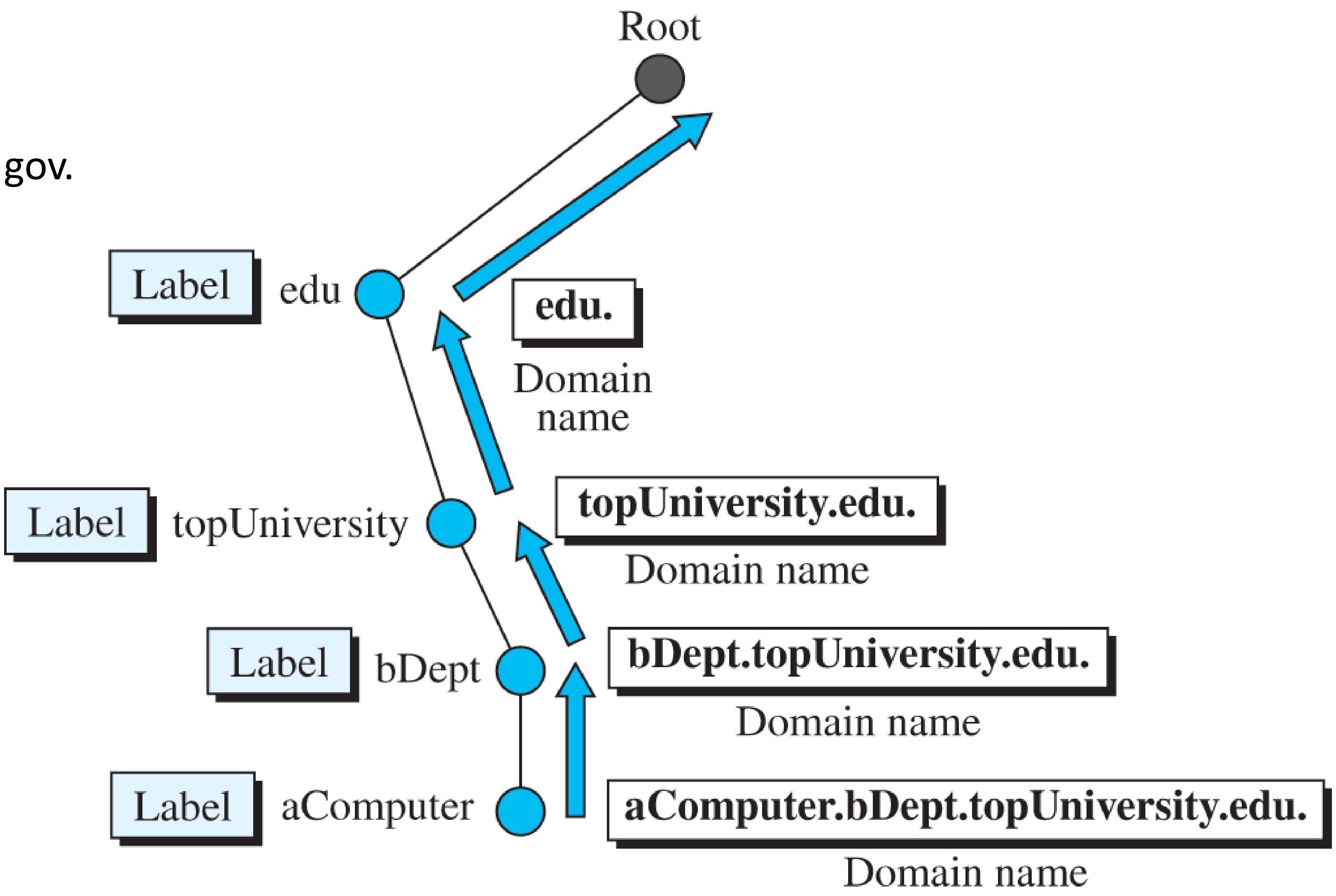
DNS Structure

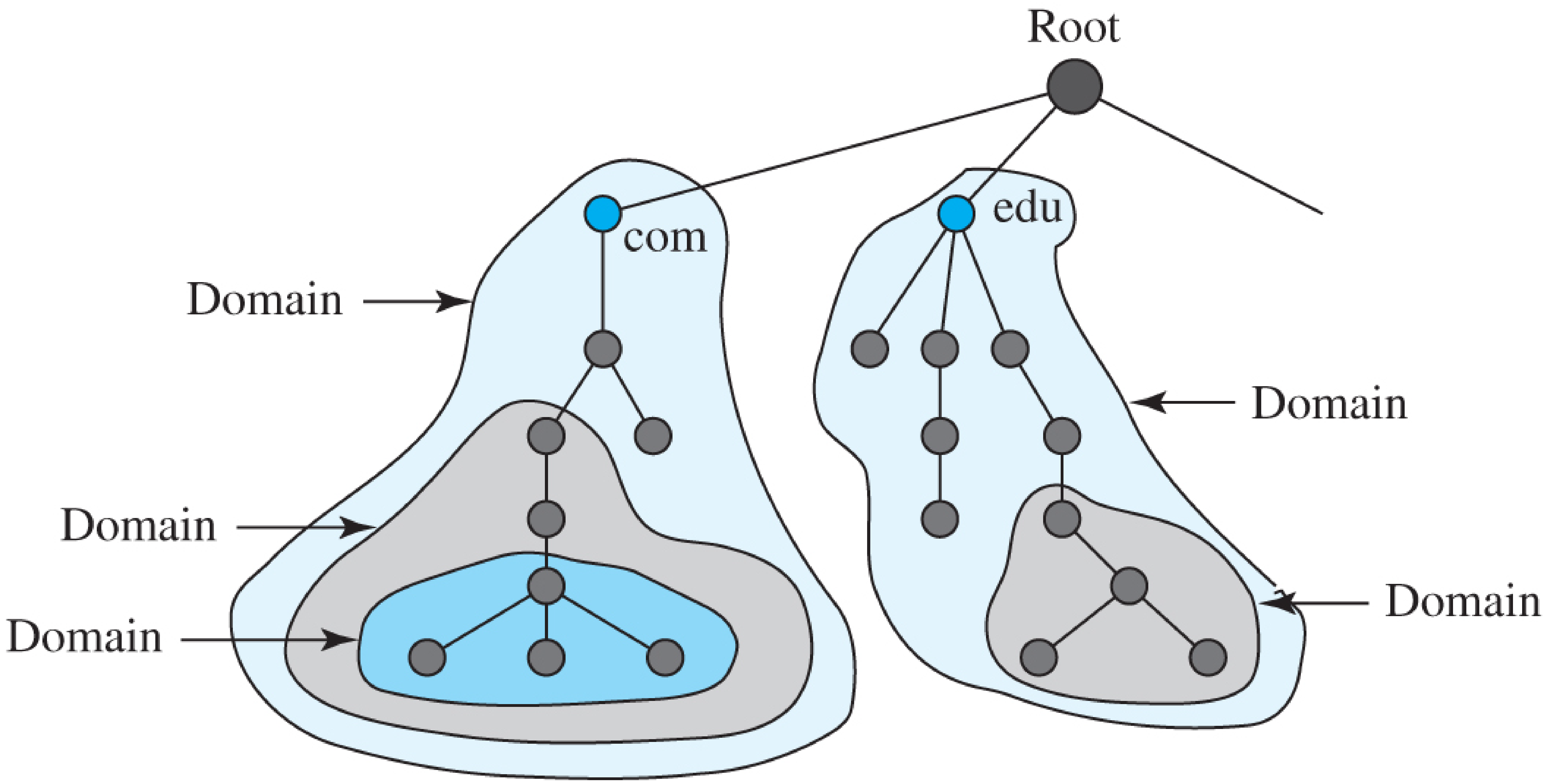
- Organized as tree: root → TLD → domain → host.
- Resolvers query name servers.



DNS Domains

- Generic domains: com, org, edu, gov.
- Country domains: th, jp, uk.





Root
com
edu
Domain
Domain
Domain
Domain
Domain

DNS Resolution

- Recursive: server queries others.
- Iterative: client queries step-by-step.

Figure 10.43 Recursive resolution

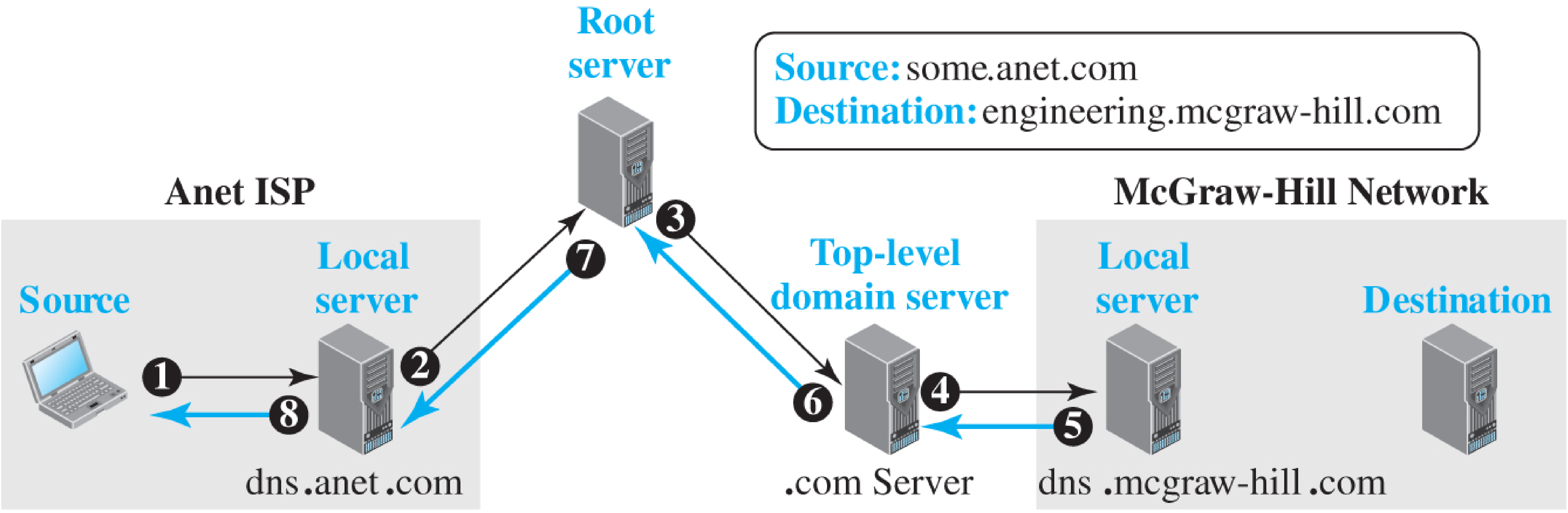
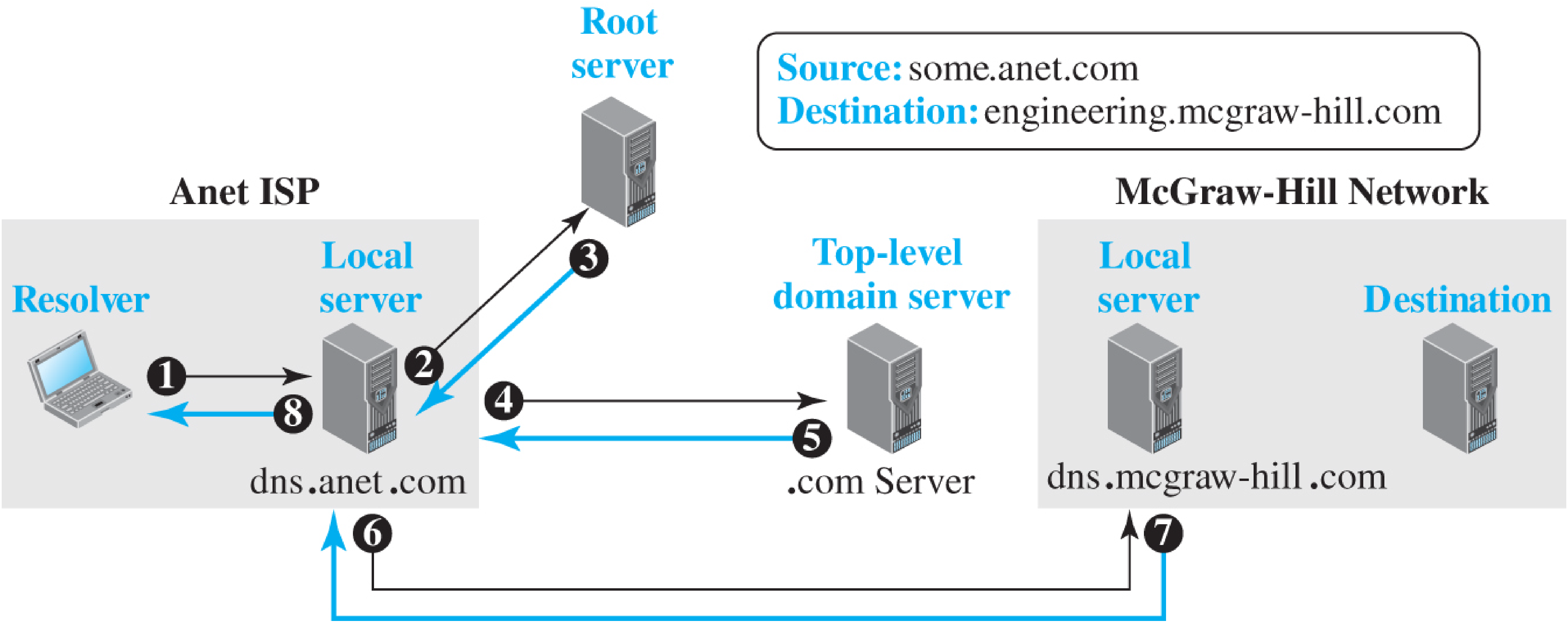


Figure 10.44 Iterative resolution



Resource Records

- (Domain, Type, Class, TTL, Value).
- Types: A, AAAA, MX, CNAME, NS, SOA.
- https://tryhackme.com/room/dnsindetail

DNS Messages

- Two message types: Query and Response.
- Use UDP (port 53) or TCP for large responses.

Type	*Interpretation of value*
A	A 32-bit IPv4 address (see Chapter 7)
NS	Identifies the authoritative servers for a zone
CNAME	Defines an alias for the official name of a host
SOA	Marks the beginning of a zone
MX	Redirects mail to a mail server
AAAA	An IPv6 address (see Chapter 7)

DNS Security

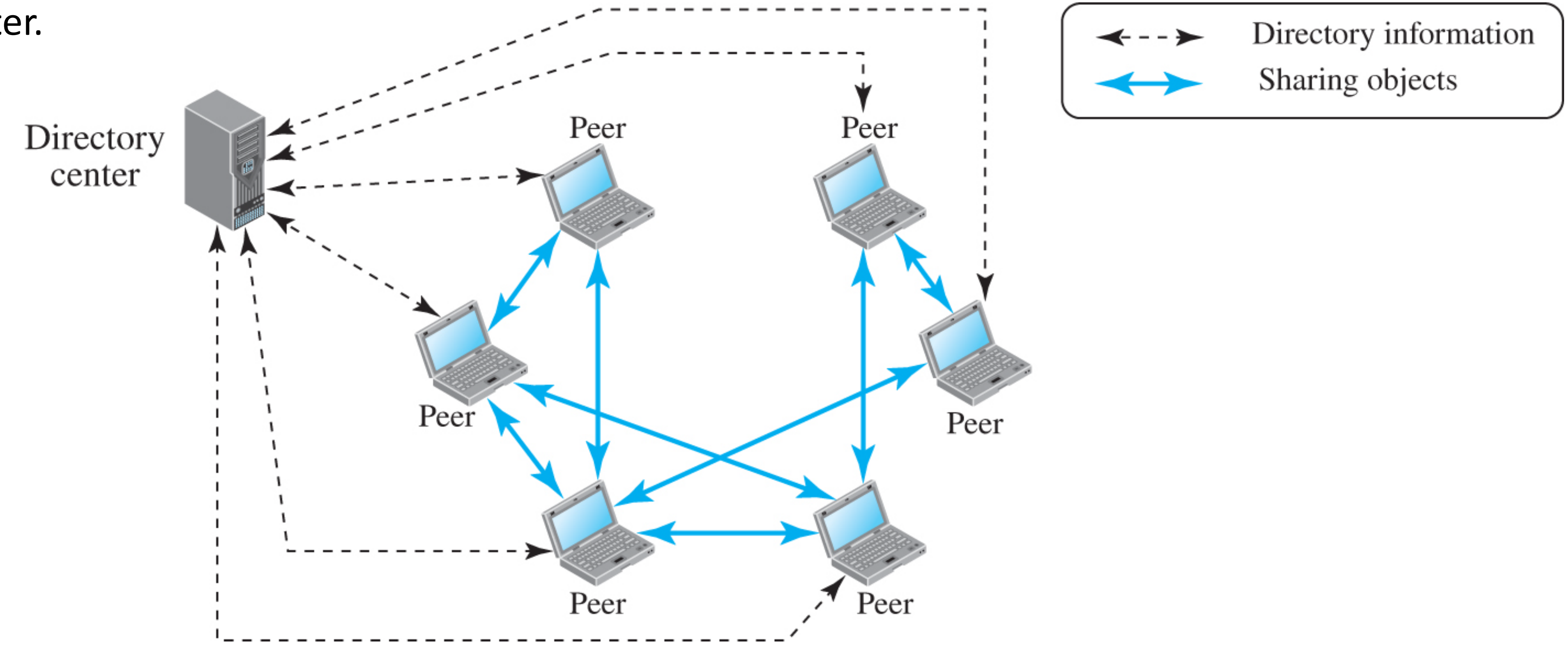
- Critical for Internet stability.
- DNSSEC adds authentication and integrity.

Peer-to-Peer Networking

- Nodes share resources directly.
- Each peer acts as both provider and consumer.

Centralized P2P

- Uses central directory.
- Transfers handled P2P.
- Example: Napster.

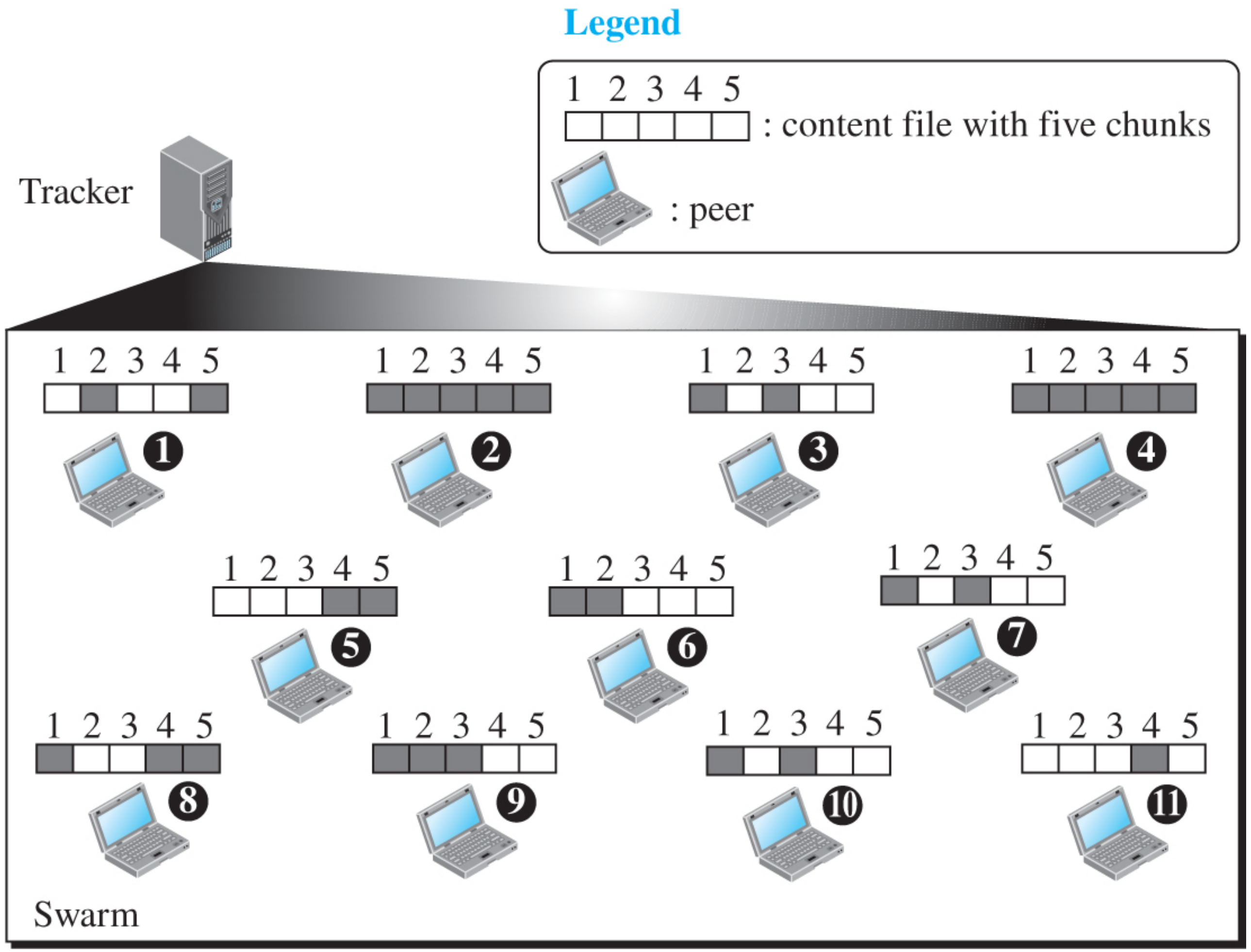


Decentralized P2P

- No central server.
- Uses overlay networks (structured/unstructured).

BitTorrent

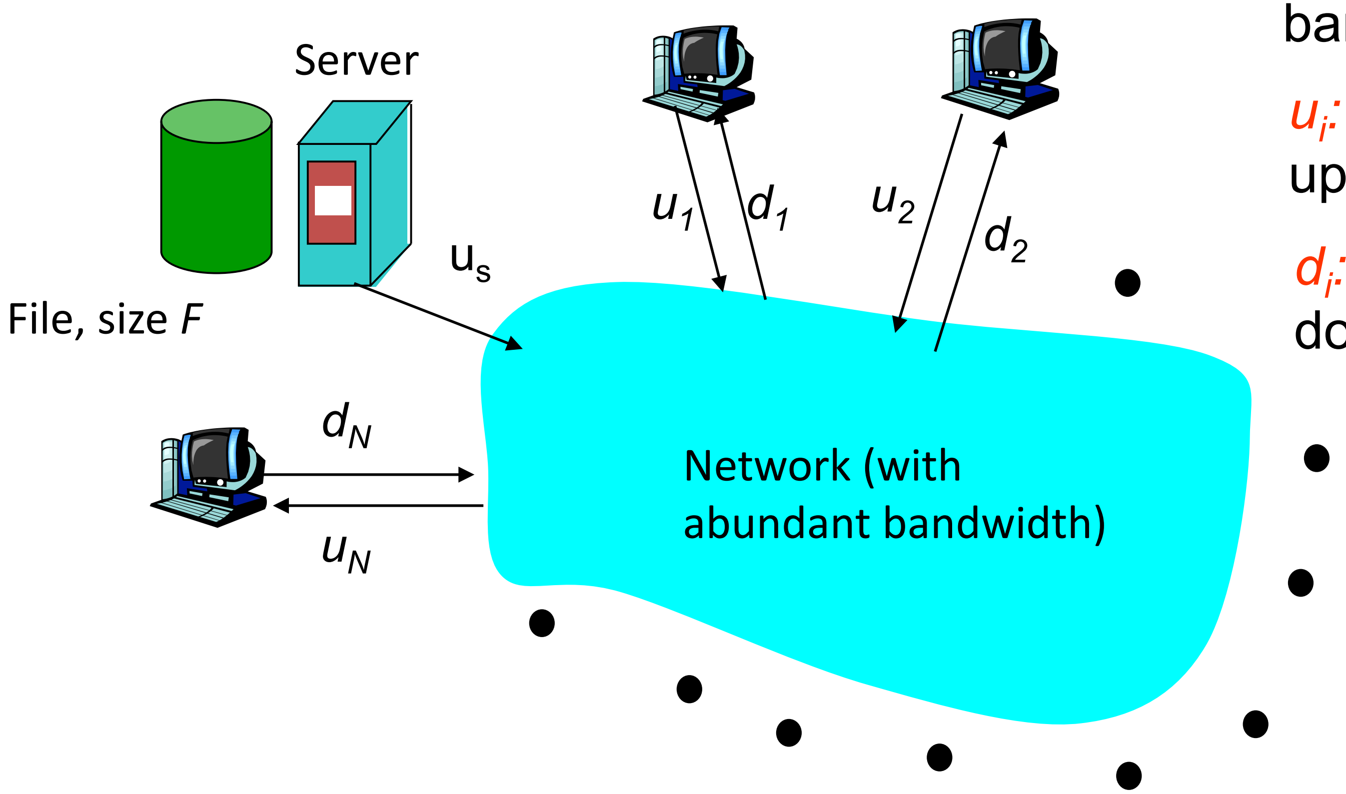
- Large file sharing using chunks.
- Peers = seeds, leechers, tracker.



Note: Peers 2 and 4 are seeds; others are leeches.

Comparing Client-server, P2P architectures

Question : How much time distribute file initially at one server to N other computers?



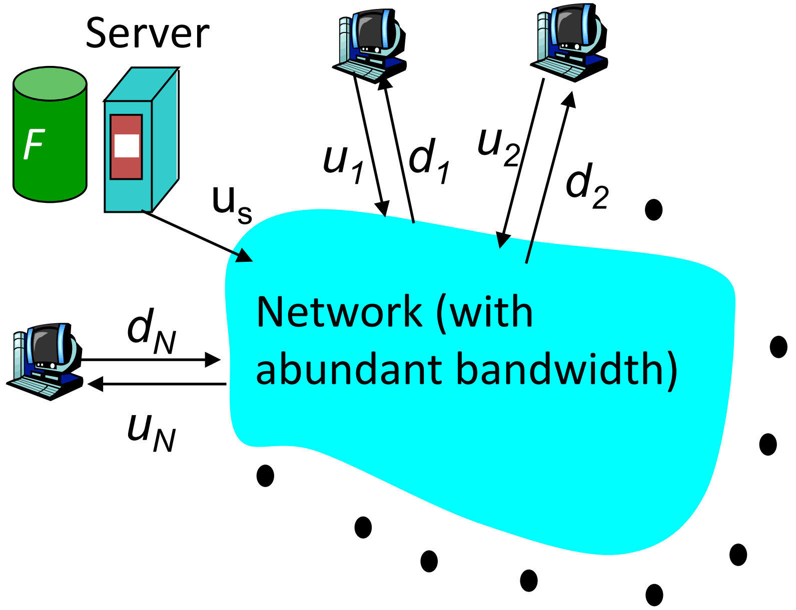
u_s: server upload bandwidth

u_i: client/peer i upload bandwidth

d_i: client/peer i download bandwidth

Client-server: file distribution time

- server sequentially sends N copies:
  - NF/u_s time
- client i takes F/d_i time to download

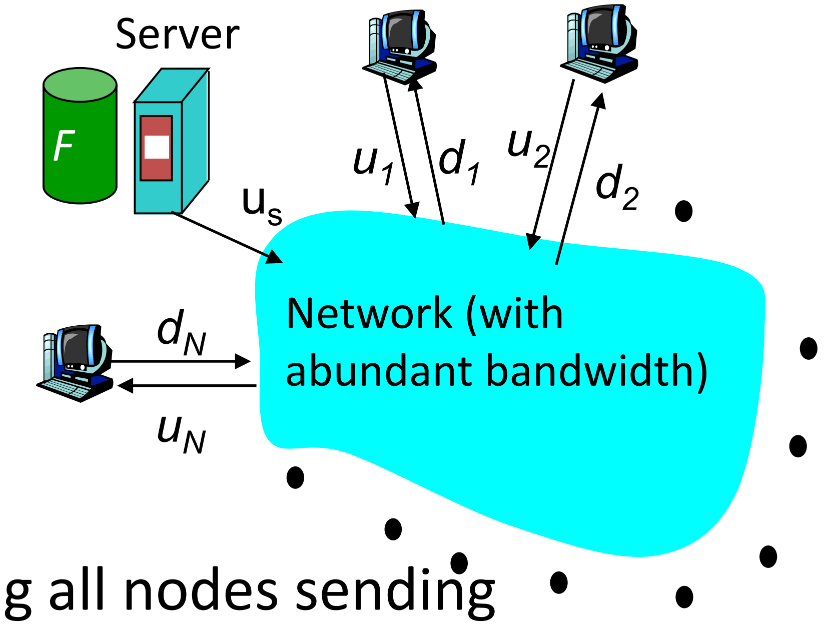


Time to distribute *F* to *N* clients using client/server approach $= d_{cs} = \max\left\{ NF/u_s,\ F/\min_i(d_i) \right\}$

increases linearly in N (for large N)

P2P: file distribution time

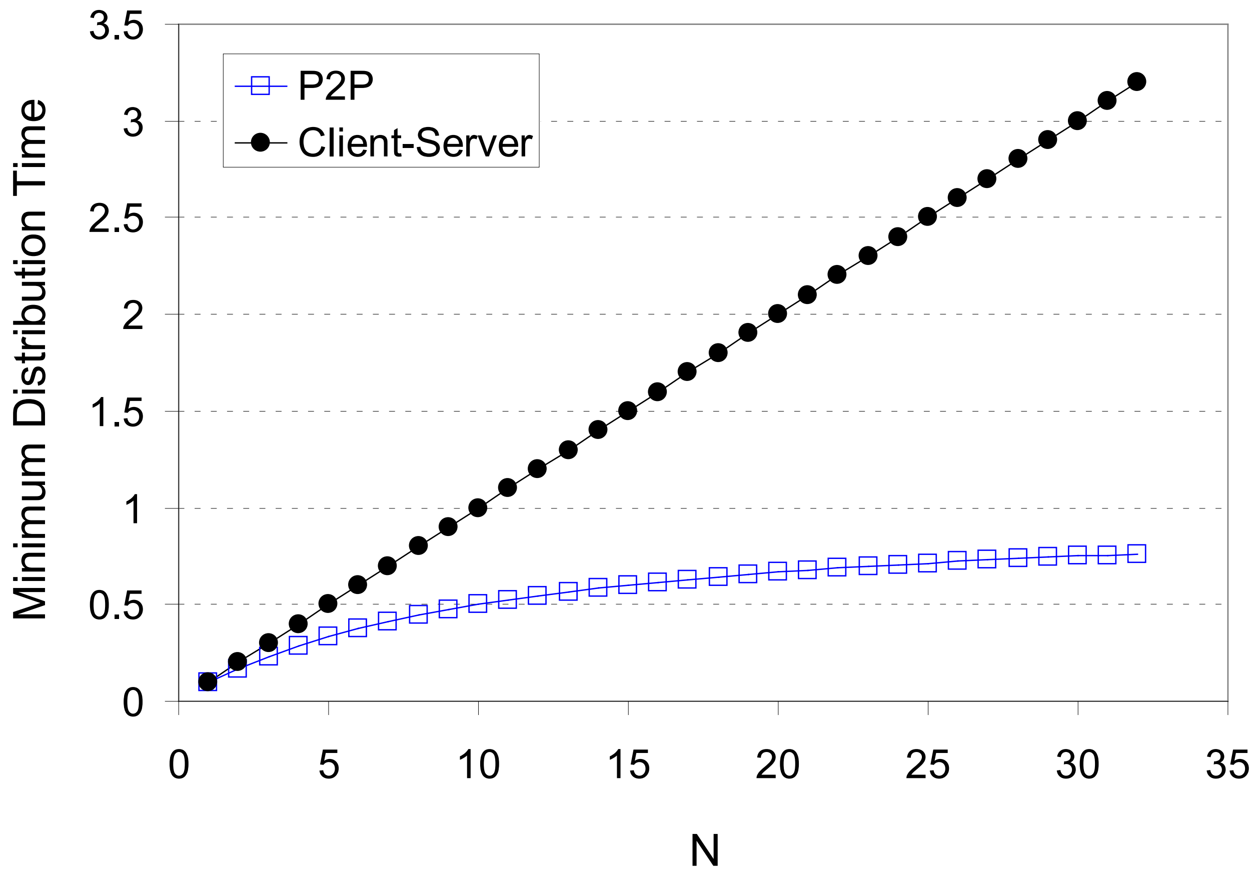
- server must send one copy: F/u_s time
- client i takes F/d_i time to download
- NF bits must be downloaded (aggregate)



❒ fastest possible upload rate (assuming all nodes sending file chunks to same peer): $u_s + \sum_{i=1,N} u_i$

$$d_{P2P} = \max \left\{ F/u_s,\ F/\min_i(d_i),\ NF/\left(u_s + \sum_{i=1,N} u_i\right) \right\}$$

Comparing Client-server, P2P architectures



Key Takeaways

- Application Layer connects users to network services.
- Includes HTTP, SMTP, DNS, and P2P models.

Because learning changes everything.®

www.mheducation.com

- © 2022 McGraw Hill, LLC. All rights reserved. Authorized only for instructor use in the classroom.
- No reproduction or further distribution permitted without the prior written consent of McGraw Hill, LLC.